8 Relaxation

Let It Slide

In 2010 Meghan Bellows was working on her PhD in chemical engineering at Princeton by day and planning her wedding by night. Her research revolved around finding the right places to put amino acids in a protein chain to yield a molecule with particular characteristics. ("If you maximize the binding energy of two proteins then you can successfully design a peptidic inhibitor of some biological function so you can actually inhibit a disease's progress.") On the nuptial front, she was stuck on the problem of seating.

There was a group of nine college friends, and Bellows agonized over who else to throw into the midst of such a mini-reunion to make a table of ten. Even worse, she'd counted up eleven close relatives. Who would get the boot from the honored parents' table, and how could she explain it to them? And what about folks like her childhood neighbors and babysitter, or her parents' work colleagues, who didn't really know anyone at the wedding at all?

The problem seemed every bit as hard as the protein problem she was working on at the lab. Then it hit her. It *was* the problem she was working on at the lab. One evening, as she stared at her seating charts, "I realized that there was literally a one-to-one correlation between the amino acids and proteins in my PhD thesis and people sitting at tables at my wedding." Bellows called out to her fiancé for a piece of paper and began scribbling equations. Amino acids became guests, binding energies became relationships, and the molecules' so-called nearest-neighbor interactions became—well—nearest-neighbor interactions. She could use the algorithms from her research to solve her own wedding.

Bellows worked out a way to numerically define the strength of the

relationships among all the guests. If a particular pair of people didn't know one another they got a 0, if they did they got a 1, and if they were a couple they got a 50. (The sister of the bride got to give a score of 10 to all the people she wanted to sit with, as a special prerogative.) Bellows then specified a few constraints: the maximum table capacity, and a minimum score necessary for each table, so that no one table became the awkward "miscellaneous" group full of strangers. She also codified the program's goal: to maximize the relationship scores between the guests and their tablemates.

There were 107 people at the wedding and 11 tables, which could accommodate ten people each. This means there were about 11^{107} possible seating plans: that's a 112-digit number, more than 200 billion googols, a figure that dwarfs the (merely 80-digit) number of atoms in the observable universe. Bellows submitted the job to her lab computer on Saturday evening and let it churn. When she came in on Monday morning, it was still running; she had it spit out the best assignment it had found so far and put it back onto protein design.

Even with a high-powered lab computer cluster and a full thirty-six hours of processing time, there was no way for the program to evaluate more than a tiny fraction of the potential seating arrangements. The odds are that the truly optimal solution, the one that would have earned the very highest score, never came up in its permutations. Still, Bellows was pleased with the computer's results. "It identified relationships that we were forgetting about," she says, offering delightful, unconventional possibilities that the human planners hadn't even considered. For instance, it proposed removing her *parents* from the family table, putting them instead with old friends they hadn't seen for years. Its final recommendation was an arrangement that all parties agreed was a hit—although the mother of the bride couldn't resist making just a few manual tweaks.

The fact that all the computing power of a lab at Princeton couldn't find the perfect seating plan might seem surprising. In most of the domains we've discussed so far, straightforward algorithms could guarantee optimal solutions. But as computer scientists have discovered over the past few decades, there are entire classes of problems where a perfect solution is essentially unreachable, no matter how fast we make our computers or how cleverly we program them. In fact, no one understands

as well as a computer scientist that in the face of a seemingly unmanageable challenge, you should neither toil forever nor give up, but—as we'll see—try a third thing entirely.

The Difficulty of Optimization

Before leading the country through the American Civil War, before drafting the Emancipation Proclamation or delivering the Gettysburg Address, Abraham Lincoln worked as a "prairie lawyer" in Springfield, Illinois, traveling the Eighth Judicial Circuit twice a year for sixteen years. Being a circuit lawyer meant literally making a circuit—moving through towns in fourteen different counties to try cases, riding hundreds of miles over many weeks. Planning these circuits raised a natural challenge: how to visit all the necessary towns while covering as few miles as possible and without going to any town twice.

This is an instance of what's known to mathematicians and computer scientists as a "constrained optimization" problem: how to find the single best arrangement of a set of variables, given particular rules and a scorekeeping measure. In fact, it's the most famous optimization problem of them all. If it had been studied in the nineteenth century it might have become forever known as "the prairie lawyer problem," and if it had first come up in the twenty-first century it might have been nicknamed "the delivery drone problem." But like the secretary problem, it emerged in the mid-twentieth century, a period unmistakably evoked by its canonical name: "the traveling salesman problem."

The problem of route planning didn't get the attention of the mathematics community until the 1930s, but then it did so with a vengeance. Mathematician Karl Menger spoke of "the postal messenger problem" in 1930, noting that no easier solution was known than simply trying out every possibility in turn. Hassler Whitney posed the problem in a 1934 talk at Princeton, where it lodged firmly in the brain of fellow mathematician Merrill Flood (who, you might recall from chapter 1, is also credited with circulating the first solution to the secretary problem). When Flood moved to California in the 1940s he spread it in turn to his colleagues at the RAND Institute, and the problem's iconic name first appeared in print in a 1949 paper by mathematician Julia Robinson. As

the problem swept through mathematical circles, it grew in notoriety. Many of the greatest minds of the time obsessed over it, and no one seemed able to make real headway.

In the traveling salesman problem, the question isn't whether a computer (or a mathematician) *could* find the shortest route: theoretically, one can simply crank out a list of all the possibilities and measure each one. Rather, the issue is that as the number of towns grows, the list of possible routes connecting them explodes. A route is just an ordering of the towns, so trying them all by brute force is the dreaded $O(n!)$ "factorial time"—the computational equivalent of sorting a deck of cards by throwing them in the air until they happen to land in order.

The question is: is there any hope of doing better?

Decades of work did little to tame the traveling salesman problem. Flood, for instance, wrote in 1956, more than twenty years after first encountering it: "It seems very likely that quite a different approach from any yet used may be required for successful treatment of the problem. In fact, there may well be no general method for treating the problem and impossibility results would also be valuable." Another decade later, the mood was only more grim. "I conjecture," wrote Jack Edmonds, "that there is no good algorithm for the traveling salesman problem."

These words would prove prophetic.

Defining Difficulty

In the mid-1960s, Edmonds, at the National Institute of Standards and Technology, along with Alan Cobham of IBM, developed a working definition for what makes a problem feasible to solve or not. They asserted what's now known as the Cobham–Edmonds thesis: an algorithm should be considered "efficient" if it runs in what's called "polynomial time"—that is, $O(n^2)$, $O(n^3)$, or in fact n to the power of any number at all. A problem, in turn, is considered "tractable" if we know how to solve it using an efficient algorithm. A problem we *don't* know how to solve in polynomial time, on the other hand, is considered "intractable." And at anything but the smallest scales, intractable problems are beyond the reach of solution by computers, no matter how powerful.*

This amounts to what is arguably the central insight of computer science. It's possible to quantify the difficulty of a problem. And some problems are just…*hard*.

Where does this leave the traveling salesman problem? Curiously enough, we are still not quite sure. In 1972, Berkeley's Richard Karp demonstrated that the traveling salesman problem is linked to a controversially borderline class of problems that have not yet been definitively proven to be either efficiently solvable or not. But so far there have been no efficient solutions found for any of those problems—making them effectively intractable—and most computer scientists believe that there *aren't* any to be found. So the "impossibility result" for the traveling salesman problem that Flood imagined in the 1950s is likely to be its ultimate fate. What's more, many other optimization problems—with implications for everything from political strategy to public health to fire safety—are similarly intractable.

But for the computer scientists who wrestle with such problems, this verdict isn't the end of the story. Instead, it's more like a call to arms. Having determined a problem to be intractable, you can't just throw up your hands. As scheduling expert Jan Karel Lenstra told us, "When the problem is hard, it doesn't mean that you can forget about it, it means that it's just in a different status. It's a serious enemy, but you still have to fight it." And this is where the field figured out something invaluable, something we can all learn from: how to best approach problems whose optimal answers are out of reach. How to relax.

Just Relax

> *The perfect is the enemy of the good.*
>
> —VOLTAIRE

When somebody tells you to relax, it's probably because you're uptight—making a bigger deal of things than you should. When computer scientists are up against a formidable challenge, their minds also turn to relaxation, as they pass around books like *An Introduction to Relaxation Methods* or *Discrete Relaxation Techniques*. But they don't relax themselves; they relax the problem.

One of the simplest forms of relaxation in computer science is known as **Constraint Relaxation**. In this technique, researchers remove some of the problem's constraints and set about solving the problem they *wish* they had. Then, after they've made a certain amount of headway, they try to add the constraints back in. That is, they make the problem temporarily easier to handle before bringing it back to reality.

For instance, you can relax the traveling salesman problem by letting the salesman visit the same town more than once, and letting him retrace his steps for free. Finding the shortest route under these looser rules produces what's called the "minimum spanning tree." (If you prefer, you can also think of the minimum spanning tree as the fewest miles of road needed to connect every town to at least one other town. The shortest traveling salesman route and the minimum spanning tree for Lincoln's judicial circuit are shown below.) As it turns out, solving this looser problem takes a computer essentially no time at all. And while the minimum spanning tree doesn't necessarily lead straight to the solution of the real problem, it is quite useful all the same. For one thing, the spanning tree, with its free backtracking, will never be any longer than the real solution, which has to follow all the rules. Therefore, we can use the relaxed problem—the fantasy—as a lower bound on the reality. If we calculate that the spanning tree distance for a particular set of towns is 100 miles, we can be sure the traveling salesman distance will be no less than that. And if we find, say, a 110-mile route, we can be certain it is at most 10% longer than the best solution. Thus we can get a grasp of how close we are to the real answer even without knowing what it is.

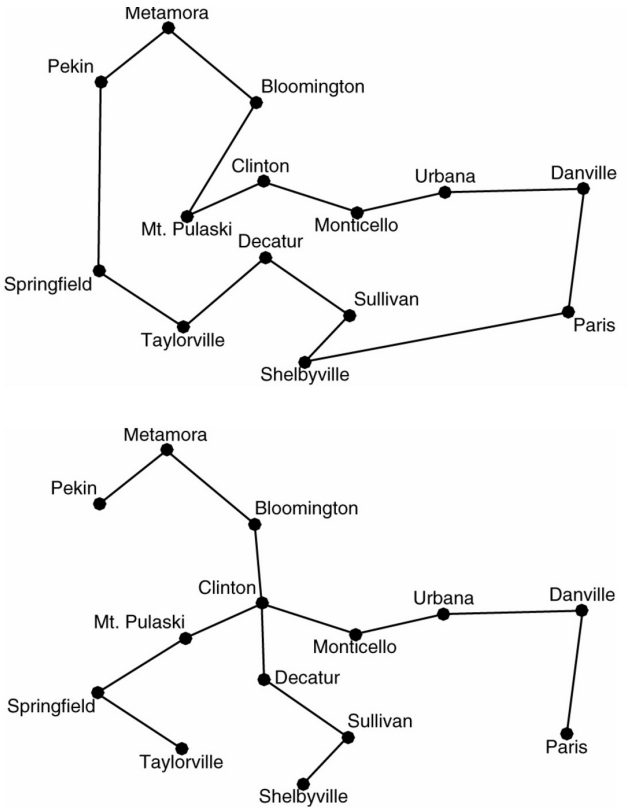


Figure 8.1 The shortest traveling salesman route (top) and minimum spanning tree (bottom) for Lincoln's 1855 judicial circuit.

Even better, in the traveling salesman problem it turns out that the minimum spanning tree is actually one of the best starting points from which to begin a search for the real solution. Approaches like these have allowed even one of the largest traveling salesman problems imaginable—finding the shortest route that visits every single town on Earth—to be solved to within less than 0.05% of the (unknown) optimal solution.

Though most of us haven't encountered the formal algorithmic version of Constraint Relaxation, its basic message is familiar to almost anyone who's dreamed big about life questions. *What would you do if you weren't afraid?* reads a mantra you might have seen in a guidance counselor's office or heard at a motivational seminar. *What would you do if you could not fail?* Similarly, when considering questions of profession or career, we ask questions like *What would you do if you won the lottery?* or, taking a different tack, *What would you do if all jobs paid the same?* The idea behind such thought exercises is exactly that of

Constraint Relaxation: to make the intractable tractable, to make progress in an idealized world that can be ported back to the real one. If you can't solve the problem in front of you, solve an easier version of it—and then see if that solution offers you a starting point, or a beacon, in the full-blown problem. Maybe it does.

What relaxation cannot do is offer you a guaranteed shortcut to the perfect answer. But computer science can also quantify the tradeoff that relaxation offers between time and solution quality. In many cases, the ratio is dramatic, a no-brainer—for instance, an answer at least half as good as the perfect solution in a quadrillionth of the time. The message is simple but profound: if we're willing to accept solutions that are close enough, then even some of the hairiest problems around can be tamed with the right techniques.

Temporarily removing constraints, as in the minimum spanning tree and the "what if you won the lottery?" examples, is the most straightforward form of algorithmic relaxation. But there are also two other, subtler types of relaxation that repeatedly show up in optimization research. They have proven instrumental in solving some of the field's most important intractable problems, with direct real-world implications for everything from city planning and disease control to the cultivation of athletic rivalries.

Uncountably Many Shades of Gray: Continuous Relaxation

The traveling salesman problem, like Meghan Bellows's search for the best seating arrangement, is a particular kind of optimization problem known as "discrete optimization"—that is, there's no smooth continuum among its solutions. The salesman goes either to this town or to that one; you're either at table five or at table six. There are no shades of gray in between.

Such discrete optimization problems are all around us. In cities, for example, planners try to place fire trucks so that every house can be reached within a fixed amount of time—say, five minutes. Mathematically, each fire truck "covers" whatever houses can be reached within five minutes from its location. The challenge is finding the minimal set of locations such that all houses are covered. "The whole [fire

and emergency] profession has just adopted this coverage model, and it's great," says University of Wisconsin–Madison's Laura Albert McLay. "It's a nice, clear thing we can model." But since a fire truck either exists at a location or it doesn't, calculating that minimal set involves discrete optimization. And as McLay notes, "that's where a lot of problems become computationally hard, when you can't do half of this and half of that."

The challenge of discrete optimization shows up in social settings, too. Imagine you wanted to throw a party for all your friends and acquaintances, but didn't want to pay for all the envelopes and stamps that so many invitations would entail. You could instead decide to mail invitations to a few well-connected friends, and tell them to "bring everyone we know." What you'd ideally want to find, then, is the smallest subgroup of your friends that knows all the rest of your social circle—which would let you lick the fewest envelopes and still get everyone to attend. Granted, this might sound like a lot of work just to save a few bucks on stamps, but it's exactly the kind of problem that political campaign managers and corporate marketers want to solve to spread their message most effectively. And it's also the problem that epidemiologists study in thinking about, say, the minimum number of people in a population—and *which* people—to vaccinate to protect a society from communicable diseases.

As we noted, discrete optimization's commitment to whole numbers—a fire department can have one engine in the garage, or two, or three, but not two and a half fire trucks, or π of them—is what makes discrete optimization problems so hard to solve. In fact, both the fire truck problem and the party invitation problem are intractable: no general efficient solution for them exists. But, as it turns out, there do exist a number of efficient strategies for solving the *continuous* versions of these problems, where any fraction or decimal is a possible solution. Researchers confronted with a discrete optimization problem might gaze at those strategies enviously—but they also can do more than that. They can try to relax their discrete problem into a continuous one and see what happens.

In the case of the invitation problem, relaxing it from discrete to continuous optimization means that a solution may tell us to send someone a quarter of an invitation, and someone else two-thirds of one.

What does that even mean? It obviously can't be the answer to the original question, but, like the minimum spanning tree, it does give us a place to start. With the relaxed solution in hand, we can decide how to translate those fractions back into reality. We could, for example, choose to simply round them as necessary, sending invitations to everyone who got "half an invitation" or more in the relaxed scenario. We could also interpret these fractions as probabilities—for instance, flipping a coin for every location where the relaxed solution tells us to put half a fire truck, and actually placing a truck there only if it lands heads. In either case, with these fractions turned back to whole numbers, we'll have a solution that makes sense in the context of our original, discrete problem.

The final step, as with any relaxation, is to ask how good this solution is compared to the actual best solution we might have come up with by exhaustively checking every single possible answer to the original problem. It turns out that for the invitations problem, Continuous Relaxation with rounding will give us an easily computed solution that's not half bad: it's mathematically guaranteed to get everyone you want to the party while sending out at most twice as many invitations as the best solution obtainable by brute force. Similarly, in the fire truck problem, Continuous Relaxation with probabilities can quickly get us within a comfortable bound of the optimal answer.

Continuous Relaxation is not a magic bullet: it still doesn't give us an efficient way to get to the truly optimal answers, only to their approximations. But delivering twice as many mailings or inoculations as optimal is still far better than the unoptimized alternatives.

Just a Speeding Ticket: Lagrangian Relaxation

> *Vizzini: Inconceivable!*
> *Inigo Montoya: You keep using that word. I do not think it means what you think it means.*
>
> —*THE PRINCESS BRIDE*

One day as a child, Brian was complaining to his mother about all the things he had to do: his homework, his chores…."Technically, you don't *have* to do anything," his mother replied. "You don't *have* to do what

your teachers tell you. You don't *have* to do what I tell you. You don't even *have* to obey the law. There are consequences to everything, and you get to decide whether you want to face those consequences."

Brian's kid-mind was blown. It was a powerful message, an awakening of a sense of agency, responsibility, moral judgment. It was something else, too: a powerful computational technique called **Lagrangian Relaxation**. The idea behind Lagrangian Relaxation is simple. An optimization problem has two parts: the rules and the scorekeeping. In Lagrangian Relaxation, we take some of the problem's constraints and bake them into the scoring system instead. That is, we take the impossible and downgrade it to costly. (In a wedding seating optimization, for instance, we might relax the constraint that tables each hold ten people max, allowing overfull tables but with some kind of elbow-room penalty.) When an optimization problem's constraints say "Do it, or else!," Lagrangian Relaxation replies, "Or else what?" Once we can color outside the lines—even just a little bit, and even at a steep cost—problems become tractable that weren't tractable before.

Lagrangian Relaxations are a huge part of the theoretical literature on the traveling salesman problem and other hard problems in computer science. They're also a critical tool for a number of practical applications. For instance, recall Carnegie Mellon's Michael Trick, who, as we mentioned in chapter 3, is in charge of scheduling for Major League Baseball and a number of NCAA conferences. What we hadn't mentioned is how he does it. The composition of each year's schedule is a giant discrete optimization problem, much too complex for any computer to solve by brute force. So each year Trick and his colleagues at the Sports Scheduling Group turn to Lagrangian Relaxation to get the job done. Every time you turn on the television or take a seat in a stadium, know that the meeting of those teams on that court on that particular night… well, it's not necessarily the optimum matchup. But it's close. And for that we have not only Michael Trick but eighteenth-century French mathematician Joseph-Louis Lagrange to thank.

In scheduling a sports season, Trick finds that the Continuous Relaxation we described above doesn't necessarily make his life any easier. "If you end up with fractional games, you just don't get anything useful." It's one thing to end up with fractional allocations of party invitations or fire trucks, where the numbers can always be rounded up if

necessary. But in sports, the integer constraints—on how many teams play a game, how many games are played in sum, and how many times each team plays every other team—are just too strong. "And so we cannot relax in that direction. We really have got to keep the fundamental [discrete] part of the model."

Nonetheless, something has to be done to reckon with the sheer complexity of the problem. So "we have to work with the leagues to relax some of the constraints they might like to have," Trick explains. The number of such constraints that go into scheduling a sports season is immense, and it includes not only the requirements arising from the league's basic structure but also all sorts of idiosyncratic requests and qualms. Some leagues are fine with the second half of the season mirroring the first, just with home and away games reversed; other leagues don't want that structure, but nonetheless demand that no teams meet for a second time until they've already met every other team once. Some leagues insist on making the most famous rivalries happen in the final game of the season. Some teams cannot play home games on certain dates due to conflicting events at their arenas. In the case of NCAA basketball, Trick also has to consider further constraints coming from the television networks that broadcast the games. Television channels define a year in advance what they anticipate "A games" and "B games" to be—the games that will attract the biggest audience. (Duke vs. UNC is a perennial A game, for instance.) The channels then expect one A game and one B game each week—but never two A games at the same time, lest it split the viewership.

Unsurprisingly, given all these demands, Trick has found that computing a sports schedule often becomes possible only by softening some of these hard constraints.

> Generally, when people first come to us with a sports schedule, they will claim…"We never do x and we never do y." Then we look at their schedules and we say, "Well, twice you did x and three times you did y last year." Then "Oh, yeah, well, okay. Then other than that we never do it." And then we go back the year before….We generally realize that there are some things they think they never do that people *do* do. People in baseball believe that the Yankees and the Mets are never at home at the same time. And it's not true. It's never been true. They are at home perhaps three games, perhaps six games in a year at the same day. But in the broad season, eighty-one games at home

for each of the teams, it's relatively rare, people forget about them.

Occasionally it takes a bit of diplomatic finesse, but a Lagrangian Relaxation—where some impossibilities are downgraded to penalties, the inconceivable to the undesirable—enables progress to be made. As Trick says, rather than spending eons searching for an unattainable perfect answer, using Lagrangian Relaxation allows him to ask questions like, "How close can you get?" Close enough, it turns out, to make everyone happy—the league, the schools, the networks—and to stoke the flames of March Madness, year after year.